

LangGraph 小白指南

把"会说话的大模型"变成"会干活的状态图"——一篇写给工科毕业生的图文解惑

开源项目类

langchain-ai/langgraph

v1.2.12 • Python

2026-10-01

学习成果：6 个疑问卡片（全部配 SVG 图解）+ 1 个 6 步成就感实验（每条命令均实际执行验证）+ 知识已入库 Obsidian vault

① 它是什么？

② 没有它之前怎么干？

③ 状态图 vs 流水线

④ 白板(State)怎么传数据

⑤ Reducer 抢白板怎么办

⑥ 条件边与循环

⑦ Checkpointer 存档器

⑧ thread_id 多用户

⑨ interrupt 人机协同

⑩ 高层 API 怎么选

⑪ 和 LangChain 什么关系

⑫ 它给我什么

动手实验 (6 步)

术语速查表

1

LangGraph 到底是什么？一句话能说清吗？

官方原话："Low-level orchestration framework for building stateful agents"——构建有状态智能体的底层编排框架。翻译成成人话：

LangGraph = 一块共享白板(State) + 一张节点路线图(Graph) + 一台存档机(Checkpointer)

你把干活的人（**Node 节点**，就是普通 Python 函数）画在路线图上，他们围着同一块白板排队干活，图负责决定"下一个谁上"，存档机负责每走一步就拍快照，断了能续。

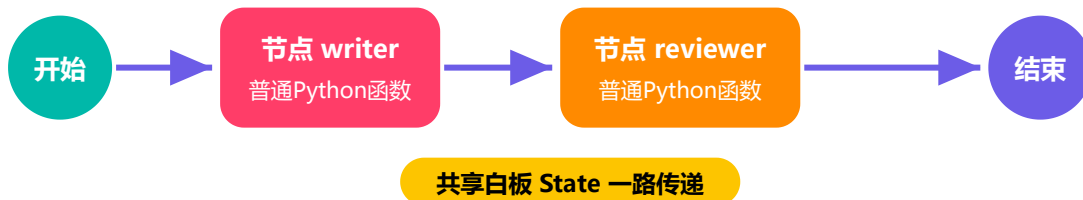


图 = 节点（函数）+ 边（顺序/条件）+ 一块一路流转的共享白板

关键认知：**节点里想调大模型就调，不想调也完全可以**。LangGraph 编排的是"任意 Python 函数"，大模型只是最常见住客。这也是本指南实验敢不接 API key 就能跑的原因。

没有 LangGraph 之前，人们怎么写 LLM 应用？痛点在哪？

2023 年大家写 Agent 基本是 "while 循环 + if 堆成山"：在一个大函数里手写循环，让模型反复"思考→调工具→把结果塞回 prompt"。代码长这样：

```
while True:
    reply = llm(messages)          # 问模型
    if "调用搜索" in reply: ...    # 手写判断
    if 用户不满意的次数 > 3: break # 手写退出
    # .....几百行 if/else 之后，没人敢动这段代码
```

四个要命的痛点：

痛点	具体惨状	LangGraph 的解法
状态没处放	对话历史、中间结果散落在闭包变量里	集中到 State 白板，由框架管理
循环写死	想加一步"人工审批"要重写半个函数	加个节点 + 条件边即可
断掉全完	跑到第 20 步进程挂了，从头再来（再花钱调 API）	Checkpoint 每步存档，断点续跑
没法画、没法查	流程只存在于 if/else 缝隙里	图结构一目了然，还能打印成流程图

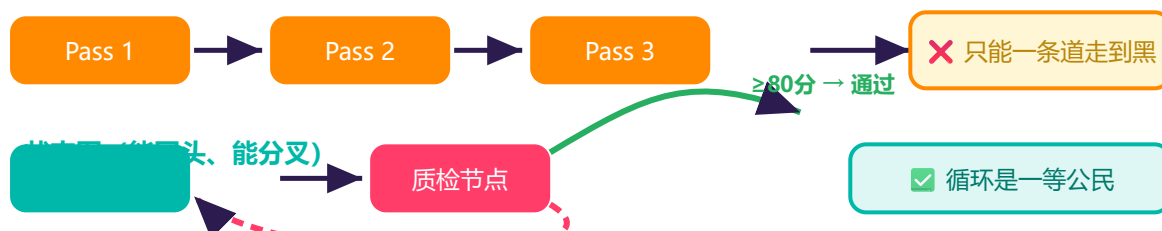
历史小故事：LangChain 团队做着做着发现，复杂的 Agent 应用真正需要的不是"更多组件"，而是一个能把**控制流显式画出来**的引擎——于是 2024 年初从 LangChain 里拆出了 LangGraph。如今 Klarna、Replit、Uber 都用它跑生产级 Agent。

"状态图"和我熟悉的"流水线 + 质检 Gate"是什么关系?

好消息：**你已经懂了 80%**。知识库里的 [[构建流水线与Pass]] 和 [[质检Gate与自我纠错循环]] 说的正是这件事——复杂工作拆成顺序阶段，阶段之间设检查站。LangGraph 就是这个思想的**工业化引擎版**：

你已学的概念	LangGraph 对应物	增强点
流水线 / Pass	Graph + Node	边可以是 条件边 ，能分叉、能回头
各 Pass 传递的中间产物	State 共享状态	不是"传来传去"，而是 所有人读写同一块白板
质检 Gate + 返工循环	Conditional Edge	循环是图的一等公民，画得出、存得了档
确定性脚本当裁判	节点函数本身	裁判也可以是真 LLM，反正都只是节点

流水线（只进不退）



上：传统流水线，单向不可回头；下：LangGraph 状态图，“打回重写”就是一条普通的边

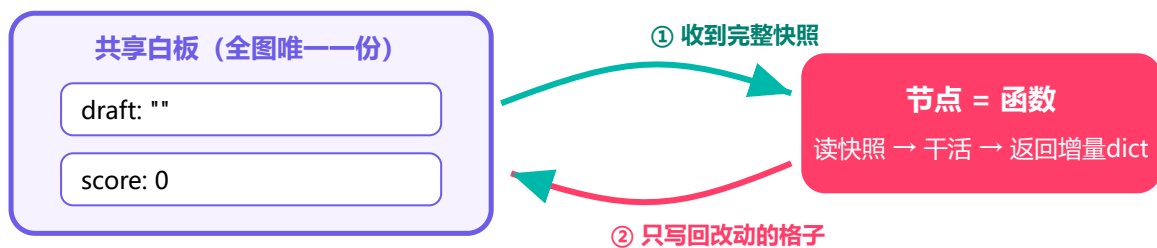
④

共享白板 State 是怎么工作的？节点之间怎么传数据？

State 就是一个 **TypedDict**（带字段类型说明的字典）。它的流转规则只有一条：

每个节点收到“整块白板的快照”，干完活只把自己改动的字段部分覆盖回去

```
class State(TypedDict):  
    draft: str    # 字段就是白板上的格子  
    score: int  
  
def writer(state): return {"draft": "新草稿"}    # 只写 draft 格子，score 不动
```



节点的输入输出协议：进 = 全量快照，出 = 部分增量

这个“进全量、出增量”的设计，让任何节点都不用关心别的节点内部怎么干活——**节点之间彻底解耦，增删节点不用改别人的代码。**

⑤

什么是 Reducer? 两个节点都想写同一格白板怎么办?

默认规则是"后写的覆盖先写的"。但想象两条分支**并行**执行后要汇合: 一个往 `messages` 格子里写了"搜索结果", 另一个写了"计算结果"——覆盖谁? LangGraph 的答案叫 **Reducer 归约器**:

在字段上声明"合并规则", 冲突就不叫冲突, 叫"合并"

```
import operator
from typing import Annotated, TypedDict

class State(TypedDict):
    # Annotated 第二个参数就是 reducer: 用 + 号合并
    messages: Annotated[list, operator.add]    # 新旧两表拼接
    score: int                                # 没写 reducer = 默认覆盖
```



并行分支汇合时, Reducer 决定怎么合: 默认覆盖, 声明 `operator.add` 则拼接

为什么重要: 后面学 `multi_agent` (多智能体) 时, 几个 Agent 并行写同一份对话历史, 全靠 `Annotated[list, operator.add]` 保证不互相覆盖。这是 **LangGraph** 面试出现率最高的细节之一。

6

条件边怎么实现"打回重写"的循环？

普通边 `add_edge("A","B")` 是死板的"A 完了必定到 B"。要"看情况走人"，用 **Conditional Edge 条件边**：

```
def gate(state) -> str:                                # 一个纯粹的裁判函数
    return "END" if state["score"] >= 80 else "writer"

g.add_conditional_edges("reviewer", gate, {"writer": "writer", "END": END})
```

裁判函数读白板 → 返回一个字符串 → 按映射表跳到对应节点。**裁判就是普通函数，所以测试它 = 测试一个纯函数，不用跑大模型**——这正是知识库 [[确定性脚本]] 思想的翻版：把"决策"从业务代码里拎出来变成可单测的小函数。



⚠ 实战红lines: **循环必须带退出条件** (比如 `attempts` 计数或最大轮数)，否则模型打分不给过就会无限打转烧钱。

7

Checkpoint 存档器是什么？"持久化/可恢复"具体指什么？

Checkpoint 干的事像游戏存档：**图每执行完一个节点，就把整块白板拍一张快照存起来**。编译时挂上即可：

```
from langgraph.checkpoint.memory import MemorySaver
app = g.compile(checkpointer=MemorySaver()) # 内存存档（学习用）
# 生产环境换成 SqliteSaver / PostgresSaver，代码一行不用改
```

官方管这叫 **Durable Execution 持久执行**：进程崩了、机器换了、隔了一天再跑，都能从最近一次快照原样恢复。这解决了 LLM 应用最贵的浪费——重跑 = 重新花钱调 API。



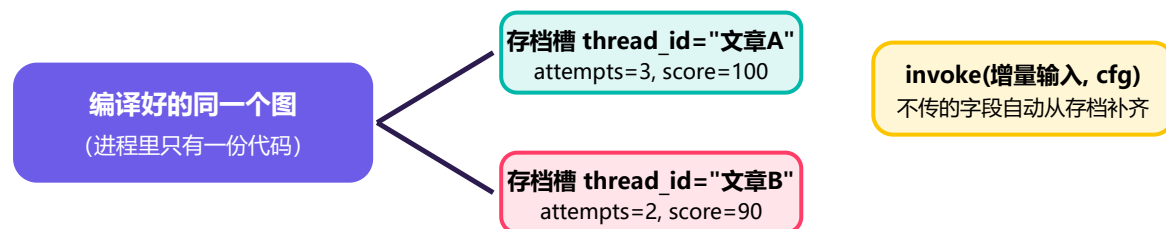
8

thread_id 是什么？为什么一个图能同时服务很多用户？

挂了 Checkpointer 之后，每次调用都要带一个 `config = {"configurable": {"thread_id": "..."}}。`

thread id 就是存档槽位号：

同一个编译好的图 + 不同 thread_id = 互不干扰的多份存档。用户 A 的对话白板在槽 "user-A"，用户 B 在槽 "user-B"，各自续各自的命。



一份图代码，N 个独立存档槽——这就是"有状态服务"的底层原理

实验步骤 4 验证过的名场面：第二轮调用只传一个新烂稿，没传 `attempts`，图从存档里补齐 `attempts=2` 接着数——用户感知就是"这服务记得我"。

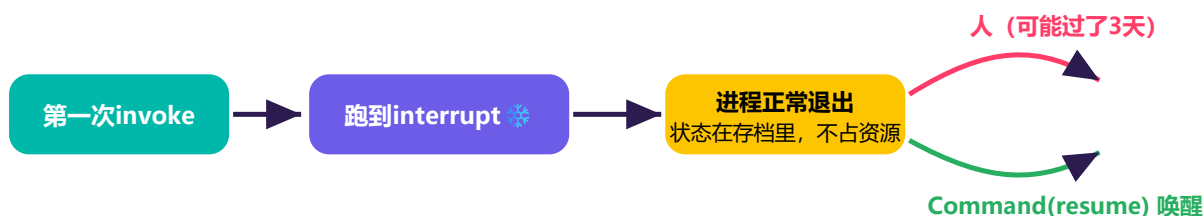
9

interrupt 人机协同：程序"暂停"了，进程去哪了？

在节点里调一句 `interrupt(问题)`，图就停在这一行，把问题抛给外面的人。魔法在于：

进程根本没有"卡住等待"——它正常退出了，状态躺在存档里。人可能一分钟后回复，也可能三天后。回复到达时，用 `Command(resume=答案)` 再 `invoke` 一次，图从暂停的那一行原样复活，连 `interrupt()` 的返回值就是人的答案。

```
def publisher(state):  
    answer = interrupt({"question": "质检通过，是否发布?"}) # 图在此冻结  
    return {"approved": answer == "yes"}  
  
r1 = app.invoke(输入, cfg) # 跑到 interrupt 停下，r1 里带 __interrupt__  
r2 = app.invoke(Command(resume="yes"), cfg) # 人审批后，从冻结点继续
```



"暂停"的真实实现 = 存档 + 退出；"恢复" = 带答案再 `invoke` 一次

场景：客服 Agent 想给客户退款 5000 元——超过权限了。这时 `interrupt` 出来问主管；主管在下班路上用手机点了同意；第二天早上 Agent 从冻结点复活继续执行。**HITL (Human-in-the-Loop) 不是"程序挂起等人"，而是"任务等待人"。**

10

prebuilt 高层 API 和 StateGraph 底层 API，我该用哪个？

LangGraph 是个三层蛋糕：

层	是什么	适合谁
Deep Agents	站在 LangGraph 之上的成品框架（会规划、会派子 Agent、带文件系统）	想要"开箱即用的深度研究 Agent"
prebuilt <code>create_react_agent</code>	一条命令生成"ReAct Agent"（模型+工具，思考→调工具→观察循环的通用配方）	快速原型、标准聊天机器人
StateGraph (本指南学的)	底层引擎：节点、边、条件边、Checkpoint、interrupt 全部亲手拼	完全掌控流程的工作流——学习阶段必须从这里入门

Deep Agents —— 成品整楼（拎包入住）

prebuilt / create_react_agent —— 精装房（一条命令的通用配方）

StateGraph 底层引擎 —— 毛坯房（每根钢筋自己铺，本指南所在层）

越高越省事越受约束；越低越费事越自由。上层建筑全部由下层搭成

判断口诀：**流程能用"模型自己决定下一步"描述 → prebuilt；流程有固定业务阶段和审批点 → 自己画 StateGraph**。两者可混用：把 create_react_agent 做的 Agent 当作大图里的一个节点。

11

LangGraph 和 LangChain 什么关系？名字里都有 Lang？

同门师兄弟，分工完全不同：

	LangChain	LangGraph
定位	组件超市：各家模型的统一接口、文档加载器、向量库适配.....	编排引擎：只管"流程怎么走、状态怎么存"
类比	食材和厨具供应商	后厨的动线设计
依赖方向	LangGraph 可以完全脱离 LangChain 使用（本指南实验就是零 LangChain 模型调用）；反过来 LangChain 1.x 的 Agent 内部用 LangGraph 做引擎	

顺带认全家族：**LangSmith**（配套的可观测/调试平台，看图执行的每一步 trace）、**Deep Agents**（上层成品）、**LangGraph Platform**（托管部署）。家族庞大但只有 LangGraph 引擎和 LangSmith 观测是日常开发绕不开的。

这个项目给我什么？我能拿它干什么？

能力清单：有状态多节点编排 / 条件分支与循环 / 每步持久化与断点恢复 / 人机协同暂停点 / 并行分支与 Reducer 合并 / 流式输出 / 一键生成流程图 / 子图嵌套（图里套图）。

结合你现有水平（会 Python、懂流水线与 Gate 思想、玩过 Three.js 项目），具体可干：

-  **给 img2threejs 那条生成流水线换引擎**：建模 Pass → 质检 Gate → 返工循环，天生就是一张 LangGraph 图，还白得断点恢复。
-  **做带记忆的客服/助手**：thread_id 一人一档，重要操作 interrupt 人工审批。
-  **多步文档处理**：抽取→核对→打分→不合格重做，质检环节继续用你的确定性脚本思想。
-  **多智能体协作**：写手 Agent、审稿 Agent、画图 Agent 各占一个节点，Reducer 管合并。
-  **Agent 评测台**：把"被测 Agent"和"考官 Agent"画进一张图，循环考到达标为止。

成就感实验：给烂文章修一条"质检返工流水线"（6 步，全程不用 API key）

目的：亲手把 ①白板 ②节点 ③条件边循环 ④流式 ⑤存档记忆 ⑥画图 ⑦人机审批 全部摸一遍。所有命令已在 Windows 11 + Python 3.14 + langgraph 1.2.12 实际执行验证。

1 搭最简图：白板 + 两个节点

怎么操作 存为 `exercise\step1_state_and_nodes.py`，运行 `PYTHONUTF8=1 python exercise/step1_state_and_nodes.py`

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    draft: str; score: int          # 白板上的两个格子

def writer(state) -> dict:          # 节点=普通函数，返回增量
    return {"draft": "LangGraph 是一个状态图编排框架。"}
def reviewer(state) -> dict:
    return {"score": 85}

g = StateGraph(State)              # 建图，声明白板格式
g.add_node("writer", writer); g.add_node("reviewer", reviewer)
g.add_edge(START, "writer"); g.add_edge("writer", "reviewer"); g.add_edge("reviewer", END)
app = g.compile()                  # 编译成可执行应用
print(app.invoke({"draft": "", "score": 0}))
```

目的 理解"节点=函数、边=顺序、白板一路传"。 **实现了什么** 最小编排：两个函数被图串起来执行。

✓ **预期结果：** 最终白板：{'draft': 'LangGraph 是一个状态图编排框架。', 'score': 85} 出现即成功——你返回的增量被合并进了最终白板。

2 上质检 Gate：条件边打出"打回重写"循环

怎么操作 存为 `exercise\step2_quality_gate_loop.py` 运行。关键三行：

```
def gate(state) -> str:
    return "END" if state["score"] >= 80 else "writer"
g.add_conditional_edges("reviewer", gate, {"writer": "writer", "END": END})
```

目的 让循环真实发生一次。 **实现了什么** 写手第一版只写 70 分，被打回，参考反馈重写到 90 分过审。

✓ **预期结果：** 「质检」打分 70 分 -> 打回重写
「写手」第 2 次写稿，参考上一轮反馈：内容太单薄，请扩充
「质检」打分 90 分 -> 通过 看到"70→打回→90→通过"即成功。

3 stream(): 把执行过程全程直播

怎么操作 把 `app.invoke(...)` 换成 `for update in app.stream(输入): print(update)` (完整代码 `exercise\step3_stream.py`)。

目的 看清图的执行节奏。 **实现了什么** 每完成一个节点立刻吐出白板增量, 不再只给最终答案。

✓ **预期结果:**

```
--- 第 1 步完成 --- {'writer': {'draft': '第1版草稿', 'score': 70, ...}}
--- 第 2 步完成 --- {'reviewer': {'feedback': '内容太单薄...'}}
--- 第 3 步完成 --- {'writer': {'draft': '第2版草稿', 'score': 90, ...}}
--- 第 4 步完成 --- {'reviewer': {'feedback': ''}} 注意输出以“节点名: 增量”为单位。
```

4 MemorySaver 存档: 跨调用记住你

怎么操作 编译时挂存档器, 两次调用共用一个 `thread_id` (完整代码 `exercise\step4_memory.py`) :

```
app = g.compile(checkpointer=MemorySaver())
cfg = {"configurable": {"thread_id": "文章A"}}
app.invoke({全量初始白板}, cfg) # 第一轮
app.invoke({"draft": "", "score": 0, "feedback": ""}, cfg) # 第二轮: 不传 attempts!
```

目的 验证"存档补齐"。 **实现了什么** 第二轮没传 `attempts`, 图从存档补出 `attempts=2`, 写手直接写第 3 稿。

✓ **预期结果:**

```
[写手] 接到白板 attempts=2, 开始第 3 次写稿
attempts 没传却自己累加到: 3 <- 说明它记得上次存档
```

5 draw_mermaid: 一行打印你的流程图

怎么操作 `print(app.get_graph().draw_mermaid())` (完整代码 `exercise\step5_draw_graph.py`) , 把输出粘到 `mermaid.live` 看图。

目的 把脑子里"想象的图"和真实图对答案。 **实现了什么** 得到一张可渲染的流程图源码, 虚线边就是条件边。

✓ **预期结果:**

```
graph TD;
    start --> writer;
    reviewer -. END .-> end ;
    reviewer -. writer; <- 这条虚线就是"打回重写"
    writer --> reviewer;
```

6 interrupt: 质检过了，先等人审批再发布

怎么操作 在发布节点里加 `interrupt()`，两次 `invoke` 完成暂停与唤醒（完整代码 `exercise\step6_human_in_loop.py`）：

```
from langgraph.types import interrupt, Command
def publisher(state):
    answer = interrupt({"question": "质检通过，是否发布?"}) # 冻结点
    return {"approved": answer == "yes"}
r1 = app.invoke(初始白板, cfg) # 停在 interrupt, r1["__interrupt__"] 带问题
r2 = app.invoke(Command(resume="yes"), cfg) # 人点头，从冻结点复活
```

目的 体验 LangGraph 招牌的 Human-in-the-Loop。 **实现了什么** 图在 `publisher` 里冻结→等人→恢复，`approved` 依人的回答落定。

✓ **预期结果：** 返回：{'question': '文章质检通过，是否发布?', 'draft': '第2版草稿'}
最终白板：{'draft': '第2版草稿', 'score': 90, 'attempts': 2, 'approved': True}

术语速查表（按出场顺序）

术语	一句话人话解释
StateGraph / Graph	路线图：声明白板格式后，把节点和边画上去
Node 节点	普通 Python 函数：读白板快照，返回增量 dict
State 状态	共享白板：TypedDict 定义的若干格子
Edge / Conditional Edge	固定边（必经之路）/ 条件边（裁判函数按白板内容决定下一站）
compile()	把画好的图编译成可执行应用 app
invoke / stream	跑一次拿最终白板 / 每步直播白板增量
Reducer	格子上的合并规则（如 <code>Annotated[list, operator.add]</code> ），并行写入不冲突
Checkpointter	存档机：每步快照，支持断点恢复； <code>MemorySaver/SqliteSaver/PostgresSaver</code> 可插拔
thread_id	存档槽位号：一个图服务多用户的关键
interrupt / Command(resume)	冻结点（等人）/ 带答案唤醒
create_react_agent	prebuilt 高层配方：一条命令生成"思考→调工具→观察"Agent
Durable Execution	持久执行：崩了从快照续跑，不重花钱
Pregel	LangGraph 内核名，借鉴 Google Pregel 论文的图计算模型（目录 <code>libs/pregel</code> ）